

International University

School of Computer Science and Engineering



Artificial Intelligence

Lab 6 & 7 Report: Naive Bayes SMS Spam Filter

Semester: 1, Year: 2025 - 2026
Lab Instructor: Dr. Ly Tu Nga

Full name: Võ Trí Khôi
Student ID: ITCSIU24045
Group: G01

1 Introduction

The objective of this lab is to implement a Naive Bayes classifier from scratch and use it as an SMS spam filter. The classifier receives a text message and predicts one of two labels: **ham**, meaning a normal message, or **spam**, meaning an unwanted advertising or scam message. The implementation is written in `naiveBayesClassifier.py`, and the experiment runner is written in `spam_filter.py`.

The lab contains four main programming problems. Problem 1 requires the implementation of the `fit()` method, which learns the vocabulary, constructs the bag-of-words table, and estimates the model parameters. Problem 2 requires the implementation of `predict_proba()`, which computes the posterior probability of each class. Problem 3 requires the implementation of `predict()`, which returns the most likely class label. Problem 4 requires the implementation of `score()`, which measures the number of correct predictions divided by the total number of predictions.

The main idea behind the model is that spam and ham messages often use different words. For example, spam messages frequently contain words such as **claim**, **prize**, **guaranteed**, and **150p**, while normal messages contain more conversational words. A bag-of-words Naive Bayes classifier can learn these word patterns and use them to classify unseen SMS messages.

2 Dataset Description

The dataset used in this lab is `SMSSpamCollection.csv`. It contains SMS messages and their corresponding labels. The file is loaded with two columns: `Label` and `SMS`. The `Label` column contains either **ham** or **spam**, and the `SMS` column contains the original message text.

Table 1: Dataset summary

Item	Value
Total messages	5572
Training messages	4458
Test messages	1114
Vocabulary size after preprocessing	7783
Spam word occurrences in training set	15190
Ham word occurrences in training set	57237
$P(\text{spam})$	0.134590
$P(\text{ham})$	0.865410

Table 2: Class distribution

Class	Full dataset	Training set	Test set
ham	4825	3858	967
spam	747	600	147

The dataset is imbalanced because there are many more ham messages than spam messages. This is realistic for spam filtering, but it also means that accuracy alone may

not tell the full story. For that reason, the evaluation section reports precision, recall, F1-score, confusion matrices, and examples of predicted probabilities.

3 Data Preprocessing

The preprocessing steps are implemented in `spam_filter.py`. First, the dataset is randomly shuffled using `random.state=1` so that the experiment is reproducible. Then the dataset is split into 80% training data and 20% test data. The resulting split contains 4458 training messages and 1114 test messages.

Each SMS message is cleaned using the following steps:

1. Remove punctuation and non-word characters using the regular expression `\W`.
2. Convert all characters to lowercase.
3. Split each message by whitespace to produce a list of words.

For example, a raw message such as

```
Free entry in 2 a wkly comp to win FA Cup final
```

is converted into a list of lowercase tokens:

```
[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final].
```

This representation is suitable for the bag-of-words model because the classifier only needs word counts, not the original sentence structure.

4 Naive Bayes Theory

The classifier is based on Bayes' theorem. For a message represented by a sequence of words

$$x = (x_1, x_2, \dots, x_n),$$

the goal is to choose the class k with the largest posterior probability:

$$\arg \max_{k \in \{\text{ham}, \text{spam}\}} P(C = k \mid x).$$

Using Bayes' theorem and dropping the denominator $P(x)$ because it is the same for all classes, the decision rule becomes:

$$\arg \max_{k \in \{\text{ham}, \text{spam}\}} P(C = k)P(x \mid C = k).$$

The Naive Bayes assumption says that each word is conditionally independent given the class. Therefore,

$$P(x \mid C = k) = \prod_{i=1}^n P(x_i \mid C = k).$$

The final classification rule is:

$$\arg \max_{k \in \{\text{ham}, \text{spam}\}} P(C = k) \prod_{i=1}^n P(x_i | C = k).$$

The prior probabilities are estimated from the training set:

$$P(C = \text{spam}) = \frac{N_{\text{spam}}}{N_{\text{samples}}}, \quad P(C = \text{ham}) = \frac{N_{\text{ham}}}{N_{\text{samples}}}.$$

The word likelihoods are estimated using word counts:

$$P(w | C = k) = \frac{\text{count}(w, C = k)}{\text{total words}(C = k)}.$$

The implementation uses Laplace smoothing to avoid zero probability when a word does not appear in one class:

$$P(w | C = k) = \frac{\text{count}(w, C = k) + \alpha}{\text{total words}(C = k) + \alpha|V|},$$

where $\alpha = 1$ and $|V|$ is the vocabulary size.

5 Problem 1: Implementing `fit()`

The purpose of the `fit()` method is to train the Naive Bayes model. It receives two inputs: `X`, the cleaned training messages, and `y`, the corresponding labels. In this implementation, each message in `X` is already a list of words.

The method solves Problem 1 in four main steps.

5.1 Building the Vocabulary

The method first scans all training messages and stores every unique word in a set. The set is then sorted and saved as `self.vocabulary`. This vocabulary defines the feature space of the model. Every word in the vocabulary becomes one possible feature.

```
vocab = set()
for message in X:
    for word in message:
        vocab.add(word)
self.vocabulary = sorted(vocab)
```

5.2 Computing Class Priors

The method counts the number of spam and ham messages in the training set. These counts are used to compute $P(\text{spam})$ and $P(\text{ham})$.

```
n_samples = len(y)
n_spam = sum(1 for label in y if label == 'spam')
n_ham = sum(1 for label in y if label == 'ham')
self.p_spam = n_spam / n_samples
self.p_ham = n_ham / n_samples
```

In this experiment, the learned priors are $P(\text{spam}) = 0.134590$ and $P(\text{ham}) = 0.865410$. This means that ham messages are much more common than spam messages in the training set.

5.3 Constructing the Bag-of-Words DataFrame

The method then constructs a DataFrame. Each row corresponds to one SMS message. Each word in the vocabulary becomes a column. The value in each cell is the number of times that word occurs in the message. A final `Label` column stores the true class.

For example, if a message contains the word `free` two times, then the `free` column for that message has value 2. If the word does not appear, the value is 0.

5.4 Estimating Word Likelihoods

After the DataFrame is built, the method separates spam rows and ham rows. It then counts how many times each word occurs in spam messages and ham messages. These counts are converted into probabilities using Laplace smoothing.

```
self.parameters_spam[word] = (
    n_word_spam + self.alpha
) / (self.n_words_spam + self.alpha * n_vocab)

self.parameters_ham[word] = (
    n_word_ham + self.alpha
) / (self.n_words_ham + self.alpha * n_vocab)
```

The learned dictionaries `self.parameters_spam` and `self.parameters_ham` are the most important parameters of the classifier. They store how likely each word is under the spam class and under the ham class.

6 Problem 2: Implementing `predict_proba()`

The purpose of `predict_proba()` is to compute the posterior probabilities for each input message. For every message, the method returns a two-column result:

$$[P(\text{ham} \mid x), P(\text{spam} \mid x)].$$

A direct multiplication of many small probabilities can cause numerical underflow. For example, multiplying hundreds of probabilities smaller than 1 may produce a number too small for normal floating-point representation. To avoid this problem, the implementation uses logarithms:

$$\log \left(P(C = k) \prod_i P(x_i \mid C = k) \right) = \log P(C = k) + \sum_i \log P(x_i \mid C = k).$$

The method starts each message with the log prior probabilities:

```
log_p_spam = np.log(self.p_spam)
log_p_ham = np.log(self.p_ham)
```

Then it adds the log likelihood of every known word:

```

for word in message:
    if word in self.parameters_spam:
        log_p_spam += np.log(self.parameters_spam[word])
        log_p_ham += np.log(self.parameters_ham[word])

```

Words that did not appear in the training vocabulary are ignored. This follows the lab suggestion that unseen words can be assigned probability 1, which means they do not change the product.

Finally, the method converts the two log scores back into normalized probabilities. The maximum log value is subtracted first to avoid overflow:

```

max_log = max(log_p_spam, log_p_ham)
p_spam_x = np.exp(log_p_spam - max_log)
p_ham_x = np.exp(log_p_ham - max_log)
total = p_spam_x + p_ham_x
proba.append([p_ham_x / total, p_spam_x / total])

```

7 Problem 3: Implementing predict()

The `predict()` method converts probabilities into final class labels. It calls `predict_proba()` and compares the two posterior probabilities. If the spam probability is larger than the ham probability, the predicted label is `spam`. Otherwise, the predicted label is `ham`.

```

for p_ham_given_x, p_spam_given_x in self.predict_proba(X):
    if p_spam_given_x > p_ham_given_x:
        predictions.append('spam')
    else:
        predictions.append('ham')

```

This method directly implements the argmax decision rule from the Naive Bayes formula.

8 Problem 4: Implementing score()

The `score()` method compares the predicted labels with the true labels. It counts how many predictions are correct and divides by the total number of labels.

```

matched = sum(1 for t, p in zip(true_list, pred_list) if t == p)
return matched / len(true_list)

```

The lab document refers to this value as recall. However, the implemented formula is actually accuracy:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}.$$

This distinction is important because recall for a class is defined differently:

$$\text{Recall}_{\text{class}} = \frac{\text{True Positives}_{\text{class}}}{\text{True Positives}_{\text{class}} + \text{False Negatives}_{\text{class}}}.$$

Therefore, the report includes both the lab score and class-specific precision, recall, and F1-score.

9 Result and Evaluation

The updated `spam_filter.py` script was extended to produce all evaluation tables in this section. It computes dataset statistics, class distributions, training and testing metrics, confusion matrices, predicted probability examples, top indicative words, and an optional comparison with `sklearn.naive_bayes.MultinomialNB` when scikit-learn is installed.

9.1 Overall Accuracy

Table 3: Overall model accuracy

Dataset	Accuracy from <code>score()</code>
Training set	0.992598
Test set	0.988330

The model achieves 99.26% accuracy on the training set and 98.83% accuracy on the test set. The training score is slightly higher than the test score, which is expected because the model learned its probabilities from the training data. The small gap between training and test accuracy suggests that the classifier generalizes well and is not heavily overfitting.

9.2 Training Set Metrics

Table 4: Training set precision, recall, and F1-score

Class	Precision	Recall	F1-score	Support
ham	0.9948	0.9966	0.9957	3858
spam	0.9781	0.9667	0.9723	600

The training results show that the model learns both classes well. The ham class has slightly better recall because it has many more examples in the training set. The spam class has recall 0.9667, meaning that the model correctly detects most spam messages but misses a small number of them.

Table 5: Training set confusion matrix

Actual class	Predicted ham	Predicted spam
ham	3845	13
spam	20	580

From the training confusion matrix, 3845 ham messages are correctly classified as ham and 580 spam messages are correctly classified as spam. There are 13 false positives, where ham messages are predicted as spam, and 20 false negatives, where spam messages are predicted as ham.

Table 6: Test set precision, recall, and F1-score

Class	Precision	Recall	F1-score	Support
ham	0.9918	0.9948	0.9933	967
spam	0.9653	0.9456	0.9553	147

9.3 Test Set Metrics

The test set results are the most important because they measure performance on unseen data. The model reaches 98.83% test accuracy. For ham messages, recall is 0.9948, meaning almost all normal messages are preserved correctly. For spam messages, recall is 0.9456, meaning the model detects 94.56% of spam messages in the test set.

Spam precision is 0.9653. This means that when the classifier predicts **spam**, it is correct 96.53% of the time. This is important for a spam filter because a false spam prediction could hide an important normal message.

Table 7: Test set confusion matrix

Actual class	Predicted ham	Predicted spam
ham	962	5
spam	8	139

The test confusion matrix gives a clearer view of the model behavior. Out of 967 ham messages, 962 are correctly classified and only 5 are incorrectly marked as spam. Out of 147 spam messages, 139 are correctly detected and 8 are missed. In a real spam filter, the 5 false positives are especially important because they represent normal messages that may be wrongly blocked. The 8 false negatives are also important because they represent spam messages that pass through the filter.

9.4 Sample Posterior Probabilities

Table 8: Sample test predictions and posterior probabilities

Index	True	Predicted	$P(\text{ham} \mid x)$	$P(\text{spam} \mid x)$	Cleaned message preview
0	ham	ham	1.000000	0.000000	later i guess i needa do mcat study too
1	ham	ham	0.999997	0.000003	but i haf enuff space got like 4 mb
2	spam	spam	0.000000	1.000000	had your mobile 10 mths update to latest orange camera video phones
3	ham	ham	0.999998	0.000002	all sounds good fingers makes it difficult to type
4	ham	ham	1.000000	0.000000	all done all handed in don t know if mega shop in

The probability examples show that the classifier often makes very confident predictions. This happens because the Naive Bayes model multiplies many word probabilities

together. When several words strongly support the same class, the posterior probability becomes very close to 0 or 1 after normalization.

9.5 Most Indicative Words

To understand what the model learned, the experiment script computes a log ratio for every word:

$$\log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}.$$

A large positive value means the word is more indicative of spam. A large negative value means the word is more indicative of ham.

Table 9: Top spam-indicative words

Word	$P(w \mid \text{spam})$	$P(w \mid \text{ham})$	Log ratio
claim	0.003961	0.000015	5.551235
prize	0.003352	0.000015	5.384181
uk	0.002525	0.000015	5.100818
150p	0.002481	0.000015	5.083427
tone	0.002220	0.000015	4.972201
guaranteed	0.001828	0.000015	4.778045
500	0.001654	0.000015	4.677961
18	0.001524	0.000015	4.595723
cs	0.001524	0.000015	4.595723
awarded	0.001436	0.000015	4.536883

These words make sense for spam classification. Many of them are related to promotions, prizes, paid text services, or claims. For example, **claim**, **prize**, **guaranteed**, and **awarded** are common in spam messages that try to persuade users to respond.

Table 10: Top ham-indicative words

Word	$P(w \mid \text{spam})$	$P(w \mid \text{ham})$	Log ratio
gt	0.000044	0.003937	-4.504802
lt	0.000044	0.003891	-4.493014
he	0.000044	0.002861	-4.185371
she	0.000044	0.002122	-3.886878
u	0.000044	0.001984	-3.819437
lor	0.000044	0.001938	-3.795907
da	0.000044	0.001738	-3.687013
later	0.000044	0.001523	-3.554745
oh	0.000044	0.001400	-3.470484
amp	0.000044	0.001246	-3.354074

The ham-indicative words are more conversational. Words such as **later**, **oh**, **he**, and **she** appear more often in normal personal messages. Some tokens, such as **gt**, **lt**, and **amp**, come from encoded text symbols in the dataset.

9.6 Comparison with Scikit-learn

The updated experiment script compares the custom classifier with `sklearn.naive_bayes.MultinomialNB`. The comparison uses `CountVectorizer` to build the word-count matrix and uses `MultinomialNB(alpha=1)` so that it matches the Laplace smoothing used in the custom model.

```
vectorizer = CountVectorizer(token_pattern=r'(?u)\b\w+\b')
X_train_counts = vectorizer.fit_transform(['_'.join(message) for message in
    X_train])
X_test_counts = vectorizer.transform(['_'.join(message) for message in X_test
    ])

sklearn_model = MultinomialNB(alpha=1)
sklearn_model.fit(X_train_counts, y_train)
sklearn_train_predictions = sklearn_model.predict(X_train_counts)
sklearn_predictions = sklearn_model.predict(X_test_counts)
```

Both models use the same cleaned tokens, the same 80/20 train-test split, and the same smoothing value $\alpha = 1$. Therefore, the comparison focuses on whether the manual implementation produces the same behavior as a standard library implementation of multinomial Naive Bayes.

Table 11: Accuracy comparison between custom model and scikit-learn model

Model	Training accuracy	Test accuracy
NaiveBayesFilter	0.992598	0.988330
MultinomialNB	0.992598	0.988330

The two models obtain the same training accuracy and the same test accuracy. This result is a strong validation of the custom implementation because `MultinomialNB` is a mature library implementation of the same algorithm. The custom model is easier to inspect because it explicitly stores the vocabulary, class priors, and word likelihood dictionaries. The scikit-learn model is more efficient because it uses optimized sparse matrix operations instead of a dense pandas DataFrame.

Table 12: Implementation comparison between custom model and scikit-learn model

Aspect	Custom model	Scikit-learn model
Classifier	<code>NaiveBayesFilter</code>	<code>MultinomialNB</code>
Feature representation	Manual bag-of-words DataFrame	<code>CountVectorizer</code> sparse matrix
Smoothing	Manual $\alpha = 1$ Laplace smoothing	Built-in <code>alpha=1</code>
Interpretability	Direct dictionaries for word probabilities	Parameters stored in optimized arrays
Efficiency	Slower for large vocabularies	Faster and more memory efficient

10 Limitations

Although the model performs well, it has several limitations.

1. The model ignores word order. For example, it treats `not free` and `free not` as the same bag of words.
2. The model assumes conditional independence between words, which is not fully true for natural language.
3. Unknown words in test messages are ignored if they were not seen during training.
4. The dense DataFrame representation is easy to understand but inefficient for large datasets because most word-count cells are zero.
5. The lab's `score()` method is called recall, but the implemented formula is accuracy.

11 Conclusion

In this lab, a Naive Bayes SMS spam filter was implemented from scratch. The `fit()` method builds the vocabulary, creates a bag-of-words DataFrame, computes class priors, and estimates word likelihoods with Laplace smoothing. The `predict_proba()` method computes posterior probabilities using log probabilities to avoid numerical underflow. The `predict()` method chooses the label with the larger posterior probability, and the `score()` method computes the proportion of correct predictions.

The model achieved 99.26% accuracy on the training set and 98.83% accuracy on the test set. On the test set, it correctly classified 962 out of 967 ham messages and 139 out of 147 spam messages. These results show that a simple bag-of-words Naive Bayes classifier is highly effective for SMS spam filtering.